

Session 9

Pós-Graduação em Sistemas Computacionais Embebidos (PGSCE) / SwitCH Embedded

Desenvolvimento de Sistemas Embebidos (PGSCE-DSISEM) / Embedded Systems Development - 2025/2026

Jorge Estrela da Silva

LINUX KERNEL MODULE COMPILATION

- On the Raspberry Pi
 - `sudo apt install kernel-headers-rpi-v8`
- Cross compilation
 - `git clone --depth=1 https://github.com/raspberrypi/linux`
 - `cd linux`
 - `KERNEL=kernel8`
 - `make ARCH=arm64 \`
`CROSS_COMPILE=~/.arm-gnu-toolchain-15.2.rel1-x86_64-aarch64-none-linux-gnu/bin/aarch64-none-linux-gnu- \`
`bcm2711_defconfig`
 - `make ARCH=arm64 \`
`CROSS_COMPILE=~/.arm-gnu-toolchain-15.2.rel1-x86_64-aarch64-none-linux-gnu/bin/aarch64-none-linux-gnu-`
 - More details in:
 - https://www.raspberrypi.com/documentation/computers/linux_kernel.html#kernel-headers
- On both cases:
 - `sudo apt install device-tree-compiler`

OUR GPIO “LIBRARY”

```
typedef struct {
    uint32_t GPFSEL[6];    /*!< GPIO Function Select */
    uint32_t reserved1;
    uint32_t GPSET[2];     /*!< GPIO Pin Output Set */
    uint32_t reserved2;
    uint32_t GPCLR[2];     /*!< GPIO Pin Output Clear */
    uint32_t reserved3;
    uint32_t GPLEV[3];     /*!< GPIO Pin Level */
    uint32_t GPEDS[3];     /*!< GPIO Pin Event Detect Status */
    uint32_t GPREN[3];     /*!< GPIO Pin Rising Edge Detect Enable */
    uint32_t GPFEN[3];     /*!< GPIO Pin Falling Edge Detect Enable */
    uint32_t GPHEN[3];     /*!< GPIO Pin High Detect Enable */
    uint32_t GPLEN[3];     /*!< GPIO Pin Low Detect Enable */
    uint32_t GPAREN[3];    /*!< GPIO Pin Async. Rising Edge Detect */
    uint32_t GPAFEN[3];    /*!< GPIO Pin Async. Falling Edge Detect */
    uint32_t GPPUD;        /*!< GPIO Pin Pull-up/down Enable */
    uint32_t GPPUDCLK[3];  /*!< GPIO Pin Pull-up/down Enable Clock */
} bcm2835_gpio_registers_t; //incomplete list
```

OUR GPIO “LIBRARY”

```
#define RPI4_PERIPH_BASE 0xFE000000
```

```
#define RPI4_GPIO_BASE (RPI4_PERIPH_BASE + 0x200000)
```

```
void SetGPIOFunction(bcm2835_gpio_registers_t *base_addr, int GPIO,  
int functionCode);
```

```
void SetGPIOOutputValue(bcm2835_gpio_registers_t *base_addr, int GPIO,  
int outputValue);
```



FIRST EXERCISE

- Write a program that receives the GPIO number and the desired state as command line parameters:

```
#include <stdio.h>
#include <sys/mman.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdlib.h>

#include "gpio.h"

int main(int argc, char *argv[]) {
    if(argc < 3)
        exit(1);

    int pin = atoi(argv[1]);
    int state = atoi(argv[2]); //0 or 1

    int mem_fd = open("/dev/mem", O_RDWR|O_SYNC);

    bcm2835_gpio_registers_t *gpio_base =
        (bcm2835_gpio_registers_t *) mmap(
...

    SetGPIOFunction(gpio_base, pin, 1);
    SetGPIOOutputValue(gpio_base, pin, state);

    return 0;
}
```



SECOND EXERCISE: PWM GENERATOR

- Configure GPIO16 for PWM generation.
- The program shall accept the PWM duty cycle (0..100) as a command line argument

...

```
#define PWM_PIN 12
```

```
int main(int argc, char *argv[]) {  
    if(argc < 1) exit(1);
```

```
    int dc = atoi(argv[1]);
```

```
    //open /dev/mem, mmap
```

```
    SetGPIOFunction(gpio_base, PWM_PIN, ???);
```



GPIO FUNCTIONS

- Recall:
 - 3 bits per pin to select function:
 - 000 for digital input
 - 001 for digital output
 - 100 for alternate function 0
 - 101 for alternate function 1
 - 110 for alternate function 2
 - 111 for alternate function 3
 - 011 for alternate function 4
 - 010 for alternate function 5



GPIO	Pull	ALT0	ALT1	ALT2
GPIO5	High	GPCLK1	SA0	DPI_D1
GPIO6	High	GPCLK2	SOE_N / SE	DPI_D2
GPIO7	High	SPI0_CE1_N	SWE_N / SRW_N	DPI_D3
GPIO8	High	SPI0_CE0_N	SD0	DPI_D4
GPIO9	Low	SPI0_MISO	SD1	DPI_D5
GPIO10	Low	SPI0_MOSI	SD2	DPI_D6
GPIO11	Low	SPI0_SCLK	SD3	DPI_D7
GPIO12	Low	PWM0_0	SD4	DPI_D8

BCM2711 ARM PERIPHERALS: PWM

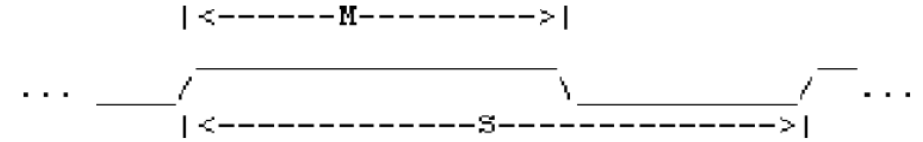
GPIO	ALT0	ALT1	ALT2	ALT3	ALT4	ALT5
GPIO12	PWM0_0					
GPIO13	PWM0_1					
GPIO18						PWM0_0
GPIO19						PWM0_1
GPIO40	PWM1_0					
GPIO41	PWM1_1					
GPIO45	PWM0_1					

BCM2711 ARM PERIPHERALS: PWM

- The PWM0 register base address is at 0xc000 relatively to the GPIO base address.

Offset	Name	Description
0x00	CTL	PWM Control
0x04	STA	PWM Status
0x08	DMAC	PWM DMA Configuration
0x10	RNG1	PWM Channel 1 Range
0x14	DAT1	PWM Channel 1 Data
0x18	FIF1	PWM FIFO Input
0x20	RNG2	PWM Channel 2 Range
0x24	DAT2	PWM Channel 2 Data

PWM CONFIGURATION



- Range register: number of clock cycles for each PWM cycle (PWM resolution)
- Data register: number of clock cycles (from 0 to range) with the PWM line set.
 - duty cycle = $\text{Data} / \text{Range}$
- Two modes:
 - M/S mode: Mark/Space mode (the “traditional” PWM approach)
 - BCM2711 PWM algorithm: distributes the “ones” and “zeros” as evenly as possible along each PWM period.

PWM CTL REGISTER

Bits	Name	Description	Type	Reset
31:16	Reserved.	-	-	-
15	MSEN2	Channel 2 M/S Enable 0: PWM algorithm is used 1: M/S transmission is used.	RW	0x0
7	MSEN1	Channel 1 M/S Enable 0: PWM algorithm is used 1: M/S transmission is used.	RW	0x0
0	PWEN1	Channel 1 Enable 0: Channel is disabled 1: Channel is enabled	RW	0x0

- **Note:** the datasheet identifies the channels as 1 and 2
 - (as opposite to the more common 0 and 1)

ACCESSING PWM REGISTERS

```
#define RPI4_PWM0_BASE (RPI4_GPIO_BASE + 0xC000)

typedef struct {
    uint32_t CONTROL;
    uint32_t STATUS;
    uint32_t DMAC[2];
    uint32_t CHN0_RANGE; //maximum value for PWM_CHANNEL0_DATA
    uint32_t CHN0_DATA;  //Pulse Width for CHANNEL0 (from 0 to PWM_CHN0_RANGE)
    uint32_t FIF1[2];   // PWM FIFO Input
    uint32_t CHN1_RANGE;
    uint32_t CHN1_DATA;
} bmc2711_pwm_registers_t;

bmc2711_pwm_registers_t *pwm0_regs = mmap(NULL, sizeof(bmc2711_pwm_registers_t),
    PROT_READ|PROT_WRITE, MAP_SHARED, mem_fd, RPI4_PWM0_BASE);
```

PWM CLOCK

- `#define BCM2835_PWM_CLOCK_BASE 0x101100`
- Set PWMCLK_CNTL to use oscillator

3:0	SRC	Clock source 0 = GND 1 = oscillator 2 = testdebug0 3 = testdebug1 4 = PLLA per 5 = PLLC per 6 = PLLD per 7 = HDMI auxiliary 8-15 = GND	RW	0x0
-----	-----	---	----	-----

SECOND EXERCISE: PWM GENERATOR

- Configure GPIO12 for PWM generation.
 - **Channel 1 must be used (CHN0 in the C struct)**
- The program shall accept the PWM duty cycle (0..100) as a command line argument

```
...
#define PWM_PIN 12

int main(int argc, char *argv[]) {
    if(argc < 1) exit(1);

    int dc = atoi(argv[1]);

    int mem_fd = open("/dev/mem", O_RDWR|O_SYNC);

    //gpio_base = mmap ...
    //pwm0_clock_regs = mmap ...
    //pwm0_regs = mmap ...

    //Set PWM clock to oscillator

    pwm0_regs->CONTROL |= (1 << 7); //set M/S mode
    pwm0_regs->CONTROL |= 1; //Enable Channel 1
    SetGPIOFunction(gpio_base, PWM_PIN, 4);

    //set range to 100 (for instance)
    ...
    //set data to dc
    ...

    return 0;
}
```





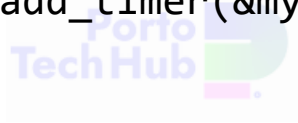
KERNEL MODULES

TIMERS

```
static struct timer_list my_timer;
static unsigned char led_status = 0;
static unsigned int blink_delay = HZ; //default config. for RPI: HZ = 250 jiffies/sec

static void my_timer_func(struct timer_list *unused){
    led_status = ~led_status;
    my_timer.expires += blink_delay;
    add_timer(&my_timer);
}

//init function
timer_setup(&my_timer, my_timer_func, 0);
my_timer.expires = jiffies + blink_delay;
add_timer(&my_timer);
```



Instituto Superior de
Engenharia do Porto



KERNEL BLINKER

- It is not adequate to use /dev/mem in kernel modules.
- Kernel space allows direct access to main memory
 - However, physical addresses must be mapped to the virtual address space used by the kernel: ioremap function

```
gpio_base = (bcm2835_gpio_registers_t *) ioremap(RPI4_GPIO_BASE,  
                                                  sizeof(bcm2835_gpio_registers_t));
```



EXTERNAL INTERRUPTS

- Create a device-tree overlay to specify desired GPIO pin
 - (on the right)
- Compile the dts file:
 - `dtc -@ -I dts -O dtb -o my_interrupt.dtbo my_interrupt.dts`
- Load the dtbo file:
 - `dtoverlay my_interrupt.dtbo`

```
/dts-v1/;  
/plugin/;
```

```
/ {  
    compatible = "brcm,bcm2835";  
  
    fragment@0 {  
        target = <&gpio>;  
        __overlay__ {  
            my_counter_int_pin: my_counter_int_pin {  
                brcm,pins = <16>;  
                brcm,function = <0>; /* Input */  
                brcm,pull = <0>; /* none */  
            };  
        };  
    };  
};
```

```
fragment@1 {  
    target-path = "/";  
    __overlay__ {  
        my_counter {  
            compatible = "my,counter";  
            interrupt-parent = <&gpio>;  
            interrupts = <16 1>; /* Pin 16, 1 for rising edge */  
            status = "okay";  
        };  
    };  
};
```

MATCHING A MODULE TO A DEVICE TREE DEVICE

- `my_probe` is called whenever a match is detected
- `my_remove` is called when the module is removed

```
//matching
```

```
static const struct of_device_id my_of_ids[] = { {  
    .compatible = "my,counter", }, { }, };
```

```
MODULE_DEVICE_TABLE(of, my_of_ids);
```

```
static struct platform_driver my_platform_driver = {  
    .probe = my_probe,  
    .remove = my_remove,  
    .driver = {  
        .name = "my_interrupt",  
        .of_match_table = my_of_ids,  
        .owner = THIS_MODULE,  
    }  
};
```

```
module_platform_driver(my_platform_driver);
```



INTERRUPT HANDLER

```
#include <linux/interrupt.h>
#include <linux/gpio/consumer.h>
#include <linux/platform_device.h>
#include <linux/of.h>
#include <linux/of_device.h>

static irqreturn_t gpio16_isr(int irq, void *data){

    return IRQ_HANDLED;
}
```



```
static int my_probe(struct platform_device *pdev) {
    int ret_val, irq;
    struct device *dev = &pdev->dev;
    dev_info(dev, "my_probe() function is called.\n");

    /* Get the Linux IRQ number */
    irq = platform_get_irq(pdev, 0); //0 is IRQ number index
    if (irq < 0){
        return -EINVAL;
    }

    /* Allocate the interrupt line */
    ret_val = devm_request_irq(dev, irq, gpio16_isr,
        IRQF_TRIGGER_RISING | IRQF_SHARED, "gpio16_int",
        dev);
    if (ret_val) {
        return ret_val;
    }

    return 0;
}
```

EXERCISE

- Write a kernel module that measures the frequency of the input signal on GPIO16.
 - The module should start counting after the first interrupt and print the number of interrupts after 1 second.
- Use **u64 ktime_get_ns()** to measure the time period.
 - `initial_time = ktime_get_ns();`
 - `u64 diff = ktime_get_ns() - initial_time;`



22

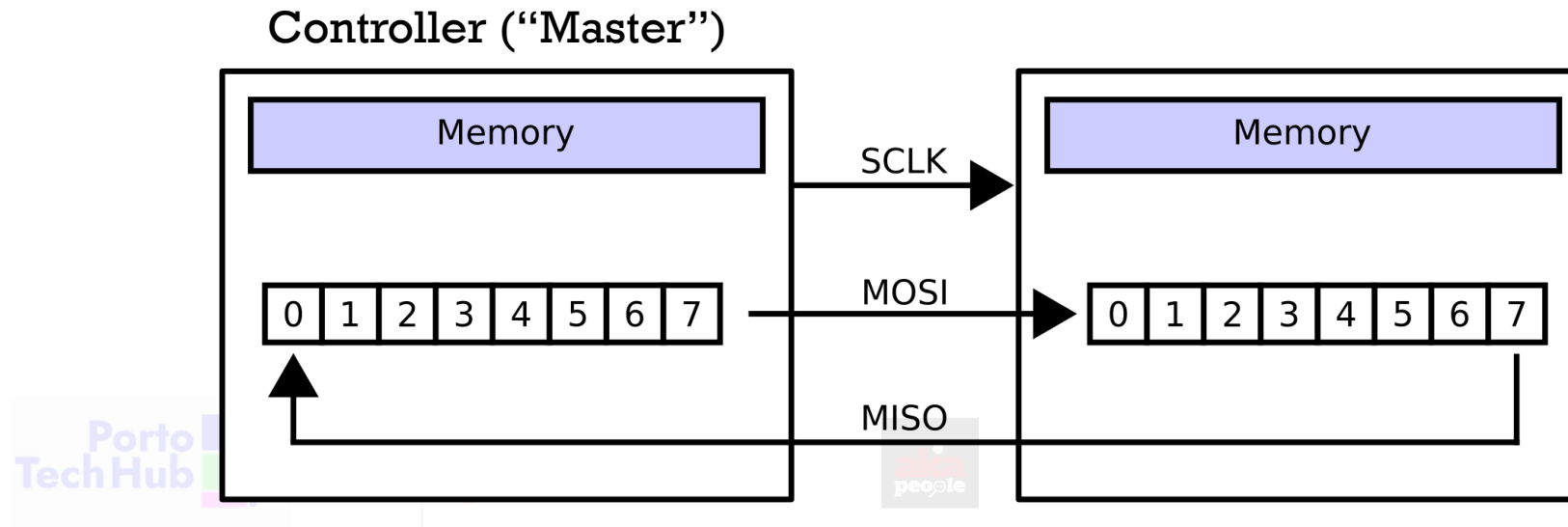
SERIAL PERIPHERAL INTERFACE (SPI)

SPI MAIN CHARACTERISTICS.

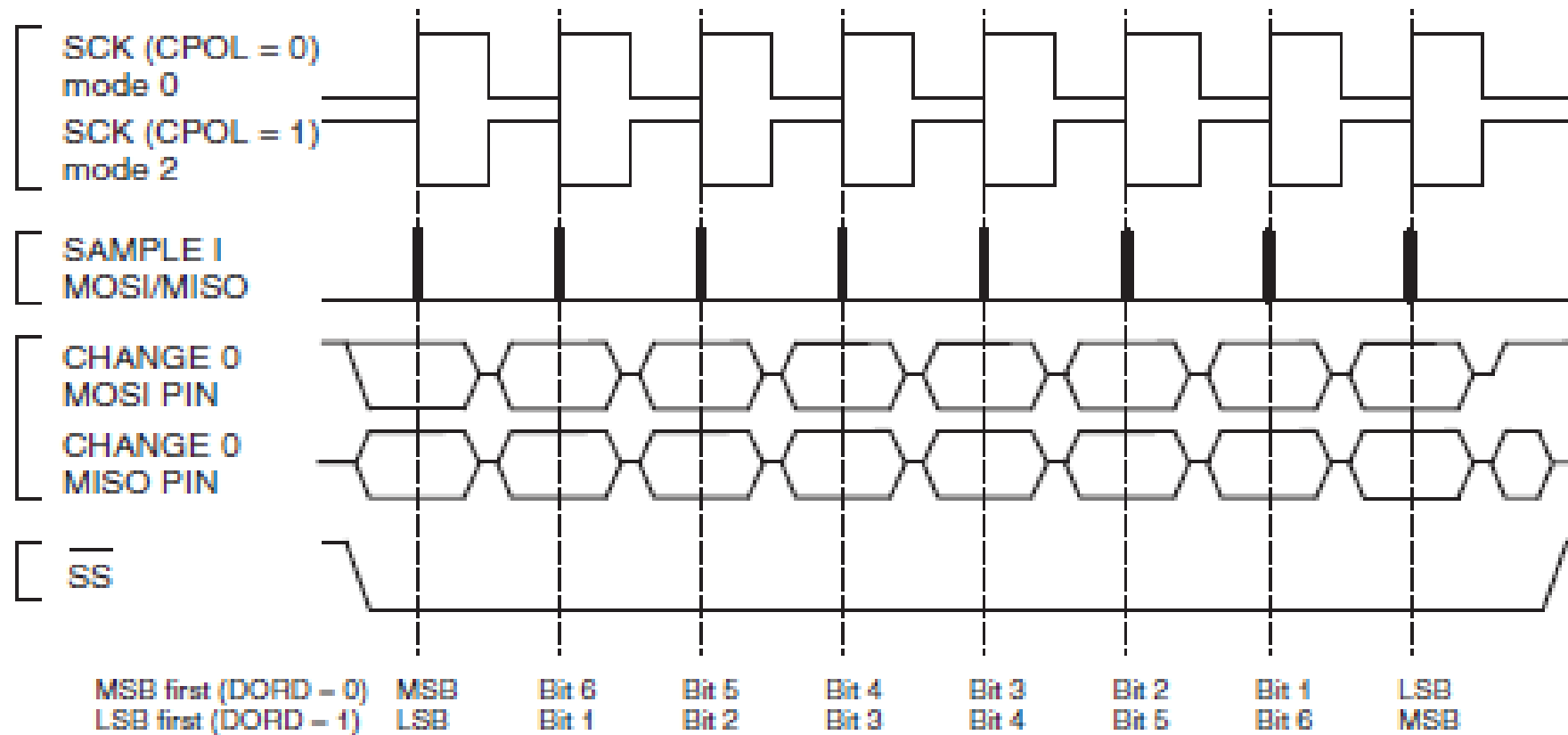
- Serial synchronous transmission
 - Clock is generated by Controller (“master”) - SCLK
- Target (“slave”) devices are selected by dedicated Slave Select (SS) Input
 - One line for each target
 - As opposed to sending address packet in I2C
- In general, it allows higher transmission speeds than I2C
 - For short distances, also
- Two data lines
 - Master Out Slave In (MOSI)
 - Master In Slave Out (MISO)
 - Allows full duplex

SPI TRANSMISSION

- At each SCLK cycle:
 - A bit is transferred from the Controller to the Target
 - A bit is transferred from Target to the Controller



CLOCK POLARITY (CPOL) AND PHASE (CPHA)



CLOCK POLARITY (CPOL) AND PHASE (CPHA)

